



UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Lab Report - 2

COURSE CODE: CSE 402.

COURSE TITLE: OPERATING SYSTEM LAB.

SUBMITTED BY:

NAME : Swagoto Utsab Singha Roy

STUDENT ID : 23101124

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C - 1

DATE : 20 - 08 - 2026

SUBMITTED TO
AITA RAHMAN ORTHI
LECTURER AT UAP

1. Problem:

Implement the **Shortest Job First (SJF – Non-Preemptive)** and **First Come First Serve (FCFS)** CPU Scheduling Algorithms for the following processes.

Table 1: Processes used for SJF

Process	Arrival Time (AT)	Burst Time (BT)
P1	2	5
P2	3	1
P3	0	2
P4	1	3
P5	1	6

Table 2: Processes used for FCFS

Process	Arrival Time (AT)	Burst Time (BT)
P1	3	5
P2	2	4
P3	4	3
P4	1	2
P5	5	3

For both SJF (Non-Preemptive) and FCFS, calculate:

- Completion Time (CT)
- Turnaround Time (TAT)
- Waiting Time (WT)
- Average Turnaround Time (AVG TAT)
- Average Waiting Time (AVG WT)

Also, represent the process execution sequence for both algorithms and compare their Average TAT and Average WT to determine which scheduling algorithm performs better for the given processes.

2. Source Code:

```
1  #shortest_job_first (non-preemptive)
2  process = ["P1", "P2", "P3", "P4", "P5"]
3  at = [2, 3, 0, 1, 1]
4  bt = [5, 1, 2, 3, 6]
5
6  n = len(process)
7  ct = [0] * n
8  wt = [0] * n
9  tat = [0] * n
10 done = [0] * n
11
12 time = 0
13 completed = 0
14
15 > while completed < n: ...
16
17 > for i in range(n): ...
18
19 sjf_avg_tat = sum(tat) / n
20 sjf_avg_wt = sum(wt) / n
21
22 > def print_table(title, headers, rows:(_iter_)): ...
23
24
25 sjf_rows = [[process[i], at[i], bt[i], ct[i], tat[i], wt[i]] for i in range(n)]
26 print_table( title: " SJF", headers: ["Process", "AT", "BT", "CT", "TAT", "WT"], sjf_rows)
27 print(f"\nAverage TAT = {sjf_avg_tat:.2f}")
28 print(f"Average WT = {sjf_avg_wt:.2f}")
29
30
```

```

51
52 lst = [
53     ["P1", 3, 5],
54     ["P2", 2, 4],
55     ["P3", 4, 3],
56     ["P4", 1, 2],
57     ["P5", 5, 3]
58 ]
59
60 lst.sort(key=lambda x: {__getitem__: x[1]})
61
62 ct2 = []
63 tat2 = []
64 wt2 = []
65 time = 0
66
67 for name, arr, burst in lst:
68     if time < arr:
69         time = arr
70     time += burst
71     ct2.append(time)
72     turnaround = time - arr
73     tat2.append(turnaround)
74     wt2.append(turnaround - burst)
75
76 avg_tat = sum(tat2) / len(tat2)
77 avg_wt = sum(wt2) / len(wt2)
78
79 fcfs_rows = [[lst[i][0], lst[i][1], lst[i][2], ct2[i], tat2[i], wt2[i]] for i in range(len(lst))]
80 print_table(title="FCFS", headers=["P_id", "AT", "BT", "CT", "TAT", "WT"], fcfs_rows)
81 print(f"\nAverage TAT = {avg_tat:.2f}")
82 print(f"Average WT = {avg_wt:.2f}")

```

```

print("\n\nCOMPARISON")
print("SJF is better in terms of TAT than FCFS" if sjf_avg_tat < avg_tat
      else "FCFS is better in terms of TAT than SJF")
print("SJF is better in terms of WT than FCFS" if sjf_avg_wt < avg_wt
      else "FCFS is better in terms of WT than SJF")

```

3. Output:

```
python3 SJF.py

SJF
Process    AT    BT    CT    TAT    WT
P1         2     5    11     9     4
P2         3     1     6     3     2
P3         0     2     2     2     0
P4         1     3     5     4     1
P5         1     6    17    16    10

Average TAT = 6.80
Average WT = 3.40

FCFS
P_id    AT    BT    CT    TAT    WT
P4       1     2     3     2     0
P2       2     4     7     5     1
P1       3     5    12     9     4
P3       4     3    15    11     8
P5       5     3    18    13    10

Average TAT = 8.00
Average WT = 4.60

COMPARISON
SJF is better in terms of TAT than FCFS
SJF is better in terms of WT than FCFS
```

4. Discussion:

In this experiment, two CPU scheduling algorithms, Shortest Job First (SJF, non-preemptive) and First Come First Serve (FCFS), were implemented and compared. It is worth noting that the SJF section of the code was tested on the process configuration in Table 1, while the FCFS section used an independently defined process list, shown in Table 2. Although the two tables specify different individual arrival times, their total burst time is identical ($2+5+1+3+6 = 5+4+3+2+3 = 17$ units), which keeps the overall workload comparable across both runs.

For SJF, the scheduler repeatedly picked whichever arrived process had the smallest remaining burst time, giving an Average TAT of 6.80 and an Average WT of 3.40. For FCFS, the processes were served strictly in order of arrival (after sorting by AT), which produced a longer Average TAT of 8.00 and a longer Average WT of 4.60, since P1 in Table 2 has a large burst time (5) and runs early, delaying the shorter jobs P3 and P5 behind it, a classic convoy effect.

Both schedules process an identical total amount of work (17 burst units) and finish within a very similar overall time frame (the SJF run ends at time 17, the FCFS run at time 18, one unit later purely because its first process arrives at $AT = 1$ instead of $AT = 0$). Yet their average TAT and WT are noticeably different. This happens because the *average* waiting time depends not on the total work done but on the *order* in which jobs are served: every process that sits in the queue behind another accumulates waiting time equal to that other

process's burst time. SJF minimises this accumulated waiting by always clearing the shortest jobs first, so fewer processes are stuck waiting behind long ones. FCFS ignores burst time completely and can place a long job in front of several short ones, so those short jobs each accumulate a large waiting time even though the total CPU-busy time is unchanged. In other words, the total “area” of work is the same, but SJF distributes the waiting more evenly and keeps it lower on average, while FCFS can concentrate a large amount of waiting onto a few unlucky short processes.

SJF is provably optimal in the sense that, for a given set of arrival times, no non-preemptive scheduling order produces a lower average waiting time than shortest-job-first. This is why it remains an important benchmark and is used whenever burst times are known or can be reliably estimated in advance, for example in batch job scheduling, print queues, or as the theoretical basis for more practical algorithms (such as multilevel feedback queues) that try to approximate SJF's benefits while limiting its downsides. Even though it is rarely used in its pure form in general-purpose operating systems, it is valuable as an ideal to be approached rather than as a directly deployable policy.

The two algorithms sit at opposite ends of the fairness spectrum. In SJF, a process with a large burst time can be repeatedly passed over if shorter processes keep arriving, so it may wait indefinitely, a phenomenon known as **starvation**. This is the direct cost of always favouring short jobs. FCFS, by contrast, cannot starve any process: every process is served strictly in the order it arrives, so every process is guaranteed to run eventually no matter how long or short it is. This is why FCFS is simple and inherently fair with respect to arrival order, but, as shown by this experiment, that fairness comes at the cost of a higher average waiting time whenever a long process happens to arrive before several short ones.

5. Conclusion:

In this experiment, SJF and FCFS CPU scheduling algorithms were implemented and compared. For the given processes, SJF achieved a lower Average TAT (6.80) and Average WT (3.40) compared to FCFS, which achieved an Average TAT of 8.00 and an Average WT of 4.60, even though both schedules covered the same total burst time and a nearly identical overall time frame. This confirms that reordering jobs by burst time, rather than by arrival time, reduces the average time processes spend waiting, since fewer short jobs are left queued behind long ones. However, SJF achieves this improvement at the cost of possible starvation for long processes, whereas FCFS guarantees that every process eventually runs, simply in arrival order, but can suffer from the convoy effect and higher average waiting times when a long job arrives early. In practice, the choice between the two reflects a trade off between minimising average waiting time (SJF) and guaranteeing fairness and freedom from starvation (FCFS).

6. GitHub:

[Click here to view source code](#)